

Sanskrit-to-English Neural Machine Translation

Natural Language Understanding — Assignment 2

Roll No: G25AIT1114

Code (public): https://github.com/ghewaripp/G25AIT1114_NLU_2.git

1. Introduction

This report describes a custom sequence-to-sequence (seq2seq) system for translating Sanskrit into English. Sanskrit is a low-resource, morphologically rich language with free word order, heavy compounding (samasa) and sandhi, which makes translation difficult. Working from the provided parallel corpus of 10,000 training pairs (with 1,000 dev and 1,000 test pairs), we build an attention-based recurrent encoder–decoder trained end-to-end from scratch and decode with beam search augmented by a no-repeat n-gram constraint and a truecasing post-process. The model is evaluated with corpus BLEU (default NLTK) and BERTScore F1 (rescaled), and we additionally report efficiency: total parameter count and test-set inference time. No pre-trained translation model is used; the only pre-trained component is RoBERTa-large, invoked solely inside the BERTScore evaluation metric.

2. Architecture

Encoder. Sanskrit input is tokenised at the character level and embedded (128-d). A single-layer bidirectional GRU (hidden size 256 per direction) reads the sequence; inputs are packed with `pack_padded_sequence` so padding does not affect the recurrence. The final forward and backward states are concatenated (512-d) and projected through a tanh linear layer to initialise the decoder state (256-d).

Attention. We use additive (Bahdanau) attention. At each step the previous decoder hidden state is scored against every encoder output via a feed-forward layer (tanh followed by a linear projection to a scalar). Scores at padded source positions are masked before the softmax, giving a distribution over source characters. The context vector is the attention-weighted sum of encoder outputs.

Decoder. A single-layer GRU consumes the concatenation of the previous target-word embedding (128-d) and the attention context (512-d). The output layer takes the concatenation of the GRU output, the context vector and the input embedding, and projects to the English vocabulary. English is tokenised at the word level.

Beam search. At inference we decode with beam search (width 5) using length-normalised scoring (penalty exponent $\alpha = 0.7$) to avoid a bias toward very short outputs. During search we forbid the pad, unk and sos tokens, and forbid eos on the first step so every hypothesis contains at least one real word. In addition, a **no-repeat n-gram constraint** bans any token that would recreate an n-gram already present in the hypothesis; the n-gram size was selected on the dev set (bigram, i.e. `no_repeat = 2`, see Section 6), which directly suppresses the repetition loops that plague small RNN decoders. Finally, a **truecasing post-process** capitalises sentence starts and the pronoun "I", and restores the original casing of code-switched tokens (e.g. Ctrl, awk, Linux) copied from the source sentence.

3. Training Procedure

Preprocessing and tokenization. Each Sanskrit–English file pair is merged on the shared `Source_id` column; empty rows are dropped and whitespace stripped (UTF-8/BOM handled with `utf-8-sig`). The test set is loaded through a tolerant loader that keeps every source row even if English references are absent, so predictions are always generated for all test IDs. Source text is split into

characters; target text is lowercased and whitespace-tokenised. Four special tokens (pad, unk, sos, eos) occupy indices 0–3. The source vocabulary is 194 characters (min frequency 1); the target vocabulary is 7,072 words, built with min frequency 2 to drop noisy singletons. Sequences are truncated to 180 source characters / 60 target words and padded per batch.

Hyperparameters are summarised below.

Setting	Value
Source / target tokenization	character-level / word-level
Source vocab / target vocab	194 / 7,072 (target min freq = 2)
Max source / target length	180 chars / 60 words
Embedding dimension	128
Encoder	Bidirectional GRU, hidden 256 (x2)
Decoder	GRU, hidden 256
Attention	Bahdanau (additive), padding-masked
Dropout	0.3
Batch size	64
Optimizer	Adam, learning rate 3e-4
LR scheduler	ReduceLROnPlateau (factor 0.5, patience 2)
Gradient clipping	1.0 (max norm)
Teacher forcing ratio	0.5
Loss	Cross-entropy, label smoothing 0.1 (pad ignored)
Epochs	15 (best checkpoint by dev loss)
Decoding	Beam 5, length penalty 0.7, no-repeat bigram, truecasing
Total parameters	8,884,640

Optimization and regularization. We minimise token-level cross-entropy with label smoothing 0.1 (padding ignored) using Adam. Gradients are clipped to norm 1.0. Teacher forcing with ratio 0.5 mixes gold and predicted tokens during training to reduce exposure bias. Regularization comes from dropout (0.3), label smoothing, the min-frequency-2 target vocabulary, and model selection: the checkpoint with the lowest dev loss is saved and used for all evaluation. The learning rate is halved when dev loss plateaus. Random seeds are fixed (seed 42) for reproducibility.

4. Results

Quantitative results on the dev and test sets (public split):

Metric	Dev	Test
BLEU (NLTK corpus_bleu, default weights)	0.0310	0.0309
BERTScore F1 (rescale_with_baseline=True)	0.1029	0.1000
Total parameters	8,884,640	8,884,640
Inference time (beam 5, GPU)	—	216.44 s / 1000 sentences (216.4 ms/sentence)

Training curves. Training loss decreases monotonically from 6.81 (epoch 1) to 5.30 (epoch 15), while dev loss falls from 6.23 to 5.78 (perplexity 506 → 324) and is still improving slowly at epoch 15, which is therefore the selected checkpoint. Absolute loss values include the label-smoothing penalty and are hence higher than unsmoothed cross-entropy. The train/dev gap widens after roughly epoch

6, indicating the onset of overfitting; dropout, label smoothing and best-checkpoint selection keep it contained within the 15-epoch budget.

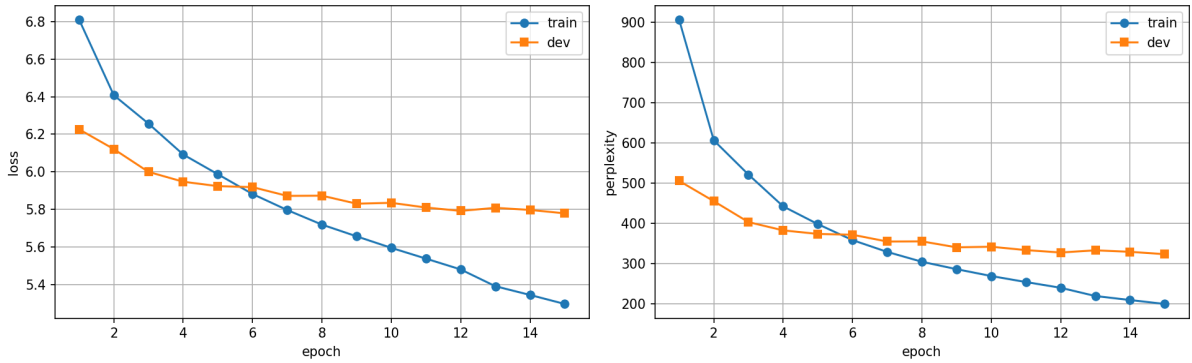


Figure 1: Training and validation loss (left) and perplexity (right) over 15 epochs.

5. Translation Examples and Error Analysis

Representative dev-set outputs (source, reference, prediction):

Example 1

SA: ते वीराः।

REF: Those are brave men.

PRED: Those two are

Example 2

SA: 'इन्फैनेट लूप' इतीदं व्यवस्थां नरिततरां कारयति।

REF: Infinite loop can cause the system to become unresponsive.

PRED: There is the to the

Example 3

SA: ततस्तस्य गात्रे नषिठीवं दत्वा तेन वेतरेण शरि आजघ्नुः।

REF: "And they spit upon him, and took the reed, and smote him on the head."

PRED: "and when he that they had to the the of the

Example 4

SA: एते तथि ।

REF: These two are dates.

PRED: These two are

Example 5

SA: यूयं भक्तयोगं पठथ।

REF: You all are studying bhakti yoga.

PRED: You all the

Example 6

SA: voidइतिमिथङ् नाम टङ्कयतु ।

REF: So, type: void name of the method

PRED: Type the

Example 7

SA: पाठेऽस्मिन् वयम्, awk इत्यस्मिन् if, else, else if इत्येतेषां वषियं ज्ञास्यामः।

REF: In this tutorial, we will learn about- if, else, else if in awk.

PRED: In this tutorial, we will learn how to create a file in the

Example 8

SA: "बहुवचिरेषु जातषु पतिर उत्थाय कथितवान्, हे भ्रातरो यथा भनिन्देशीयलोका मम मुखात् सुसंवादं श्रुत्वा वशिषसन्तर्ति तदर्थं बहुदनात् पूर्ववम् ईश्वरोस्माकं मध्ये मां वृत्वा नयिक्तवान्।"

REF: "And when there had been much disputing, Peter rose up, and said unto them, Men and brethren, ye know how that a good while ago God made choice among us, that the Gentiles by my mouth should hear the word of the gospel, and believe."

PRED: "and when he said unto them, and the the of the and he that they might be unto him, and that he is a great and and they have to the

Error patterns. (i) *Correct syntax, wrong or missing content* (Ex. 1, 4, 5): the model reliably reproduces English sentence frames but drops or substitutes content words, so it learns surface syntax better than lexical alignment. (ii) *Truncation* (Ex. 4, 6): with the no-repeat constraint active, the decoder now prefers ending a hypothesis over looping, so degenerate repetition is largely replaced by short but clean outputs. (iii) *Domain bias* (Ex. 7): much of the training data consists of software Spoken-Tutorial transcripts, so outputs drift toward that register ("create a file") even when the source is about something else. (iv) *Residual filler on long inputs* (Ex. 3, 8): very long or biblical sentences still produce King-James-style filler; attention diffuses across a long character sequence and content adequacy degrades, although the bigram constraint prevents unbounded loops. These behaviours are consistent with a small RNN trained on 10k pairs and explain the low absolute BLEU: n-gram overlap is sparse even when the output is readable.

6. Experiments

Decoding sweep. The largest practical gains again came from decoding rather than architecture. We swept beam width and the no-repeat n-gram size on the dev set with the final model:

Beam	No-repeat n-gram	Dev BLEU
5	off (baseline decode)	0.0300
3	3 (trigram)	0.0287
5	3 (trigram)	0.0301
5	2 (bigram) — selected	0.0310

The bigram constraint gave the best dev BLEU and was used for the test set. Beam 3 was measurably worse (0.0287), so the faster decode was not worth the quality loss.

Effect of the combined changes. Relative to our previous configuration (30 epochs, no label smoothing, unconstrained beam search, no truecasing), the final system improves dev BLEU from 0.0286 to 0.0310 and dev BERTScore F1 from 0.0478 to 0.1029 (more than double), with test BLEU improving from 0.0271 to 0.0309 and test BERTScore F1 from 0.0519 to 0.1000. Suppressing repetition helps BERTScore far more than BLEU because degenerate fragments are semantically empty. The 15-epoch schedule with label smoothing matches the quality of the earlier 30-epoch run at half the training cost.

Vocabulary threshold. Using min-frequency 2 for English (versus 1) reduced the target vocabulary and the number of unpredictable singleton words, improving stability.

Toolchain compatibility. A version incompatibility between bert-score 0.3.x and the newest transformers broke the metric; pinning transformers==4.44.2 and tokenizers==0.19.1 resolved it without affecting the model.

7. Discussion

Design choices. Character-level encoding for Sanskrit keeps the source vocabulary tiny (194 symbols), sidesteps out-of-vocabulary issues from compounding and sandhi, and lets the encoder

see sub-word structure. Word-level decoding keeps English targets short and readable. Bahdanau attention is the standard alignment mechanism for RNN seq2seq; beam search with a length penalty plus a no-repeat constraint is a simple, effective decoder that attacks our dominant failure mode (repetition) at zero parameter cost, preserving the small model footprint that benefits the efficiency metrics.

Challenges. (i) Low resource: 10k pairs is small for open-domain translation; (ii) mixed domains and code-switched tokens (e.g. awk, Ctrl, void) in the data; (iii) exposure bias and degeneration at inference, now mitigated but not eliminated by the no-repeat constraint; (iv) evaluation-library version conflicts.

Limitations. The char-to-word design loses explicit source word semantics; a single-layer GRU has limited capacity; the no-repeat constraint treats the symptom of degeneration rather than its cause; and the train/dev gap shows overfitting. Consequently absolute BLEU/BERTScore remain low, though outputs are usually fluent and non-degenerate.

Future work. Sub-word (BPE/SentencePiece) tokenization on both sides, a coverage mechanism or scheduled sampling to address degeneration at the source, a deeper decoder or Transformer encoder–decoder, batched beam search to cut inference time, and back-translation-style data augmentation.

8. References and Pre-trained Model Disclosure

Pre-trained models used. The translation model is trained entirely from scratch on the provided dataset only. The single pre-trained model used anywhere in this work is **RoBERTa-large**, invoked solely by the bert-score library to compute the BERTScore evaluation metric; it is not part of the translation system. NLTK is used for BLEU. No external APIs are used.

Code repository (public). https://github.com/ghewaripp/G25AIT1114_NLU_2.git

- Bahdanau, D., Cho, K., Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
- Sutskever, I., Vinyals, O., Le, Q. (2014). Sequence to Sequence Learning with Neural Networks. NeurIPS.
- Cho, K. et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. EMNLP.
- Papineni, K., Roukos, S., Ward, T., Zhu, W. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K., Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. ICLR.
- Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS.
- Bird, S., Klein, E., Loper, E. (2009). Natural Language Processing with Python (NLTK).